

AI LAB WEEK-2

8 PUZZLE DFS

```
def dfs_solve(initial_state, goal_state):

    def get_blank_position(state):
        for i in range(3):
            for j in range(3):
                if state[i][j] == 0:
                    return (i, j)
        return None

    def get_neighbors(state):
        neighbors = []
        r, c = get_blank_position(state)
        moves = [(0, 1), (0, -1), (1, 0), (-1, 0)]

        for dr, dc in moves:
            nr, nc = r + dr, c + dc

            if 0 <= nr < 3 and 0 <= nc < 3:
                new_state_list = [list(row) for row in state]
                new_state_list[r][c], new_state_list[nr][nc] = (
                    new_state_list[nr][nc],
                    new_state_list[r][c]
                )
                neighbors.append(tuple(tuple(row) for row in new_state_list))

        return neighbors

    stack = [(initial_state, [])]
    visited = {initial_state}
```

```
while stack:

    current_state, path = stack.pop()

    if current_state == goal_state:

        return path + [current_state]

    for neighbor in get_neighbors(current_state):

        if neighbor not in visited:

            visited.add(neighbor)

            stack.append((neighbor, path + [current_state]))

return None
```

```
def print_state(state):

    for row in state:

        print(" ".join(map(str, row)))

    print()
```

```
initial_state_dfs = (

    (1, 2, 3),

    (0, 4, 6),

    (7, 5, 8)

)
```

```
goal_state_dfs = (

    (1, 2, 3),

    (4, 5, 6),

    (7, 8, 0)

)
```

```
print("Solving 8-Puzzle with DFS:")
```

```
print("Initial State:")
print_state(initial_state_dfs)

print("Goal State:")
print_state(goal_state_dfs)

dfs_solution = dfs_solve(initial_state_dfs, goal_state_dfs)

if dfs_solution:
    print(f"DFS found a solution in {len(dfs_solution) - 1} steps:")

    for step, state in enumerate(dfs_solution):
        print(f"Step {step}:")
        print_state(state)
else:
    print("DFS: No solution found.")
```

Solving 8-Puzzle with DFS:

Initial State:

```
1 2 3
0 4 6
7 5 8
```

Goal State:

```
1 2 3
4 5 6
7 8 0
```

8 PUZZLE IDS

```
import collections
```

```
def ids_solve(initial_state, goal_state, max_depth=20):
```

```
    def get_blank_position(state):
```

```
        for i in range(3):
```

```
            for j in range(3):
```

```
                if state[i][j] == 0:
```

```
                    return (i, j)
```

```
    return None
```

```
def get_neighbors(state):
```

```
    neighbors = []
```

```
    r, c = get_blank_position(state)
```

```
    moves = [(0, 1), (0, -1), (1, 0), (-1, 0)]
```

```
    for dr, dc in moves:
```

```
        nr, nc = r + dr, c + dc
```

```
        if 0 <= nr < 3 and 0 <= nc < 3:
```

```
            new_state_list = [list(row) for row in state]
```

```
            new_state_list[r][c], new_state_list[nr][nc] = (
```

```
                new_state_list[nr][nc],
```

```
                new_state_list[r][c]
```

```
            )
```

```
            neighbors.append(tuple(tuple(row) for row in new_state_list))
```

```
    return neighbors
```

```
def dls(initial_state, goal_state, limit):
```

```
    stack = [(initial_state, [], 0)]
```

```
    visited_at_depth = collections.defaultdict(set)
```

```
visited_at_depth[0].add(initial_state)
```

```
while stack:
```

```
    current_state, path, current_depth = stack.pop()
```

```
    if current_state == goal_state:
```

```
        return path + [current_state]
```

```
    if current_depth < limit:
```

```
        for neighbor in get_neighbors(current_state):
```

```
            if neighbor not in visited_at_depth[current_depth + 1]:
```

```
                visited_at_depth[current_depth + 1].add(neighbor)
```

```
                stack.append(
```

```
                    (neighbor, path + [current_state], current_depth + 1)
```

```
                )
```

```
    return None
```

```
for depth_limit in range(max_depth + 1):
```

```
    print(f"Attempting IDS with depth limit: {depth_limit}")
```

```
    solution = dls(initial_state, goal_state, depth_limit)
```

```
    if solution is not None:
```

```
        return solution
```

```
return None
```

```
def print_state(state):
```

```
    for row in state:
```

```
        print(" ".join(map(str, row)))
```

```
    print()
```

```
initial_state_ids = (  
    (1, 2, 3),  
    (4, 0, 6),  
    (7, 5, 8)  
)
```

```
goal_state_ids = (  
    (1, 2, 3),  
    (4, 5, 6),  
    (7, 8, 0)  
)
```

```
print("\nSolving 8-Puzzle with IDS:")
```

```
print("Initial State:")
```

```
print_state(initial_state_ids)
```

```
print("Goal State:")
```

```
print_state(goal_state_ids)
```

```
ids_solution = ids_solve(initial_state_ids, goal_state_ids, max_depth=10)
```

```
if ids_solution:
```

```
    print(f"IDS found a solution in {len(ids_solution) - 1} steps:")
```

```
    for step, state in enumerate(ids_solution):
```

```
        print(f"Step {step}:")
```

```
        print_state(state)
```

```
else:
```

```
    print("IDS: No solution found within the specified maximum depth.")
```

...

Solving 8-Puzzle with IDS:

Initial State:

1 2 3
4 0 6
7 5 8

Goal State:

1 2 3
4 5 6
7 8 0

Attempting IDS with depth limit: 0

Attempting IDS with depth limit: 1

Attempting IDS with depth limit: 2

IDS found a solution in 2 steps:

Step 0:

1 2 3
4 0 6
7 5 8

Step 1:

1 2 3
4 5 6
7 0 8

Step 2:

1 2 3
4 5 6
7 8 0